Plus One

```
class Solution(object):
    def plusOne(self, digits):
        """
        :type digits: List[int]
        :rtype: List[int]
        """
        # Traverse the list from the last element to the first
        for i in range(len(digits)-1, -1, -1):
            # If the current digit is 9, set it to 0
            if digits[i] == 9:
                digits[i] = 0
            else:
                # If the current digit is not 9, increment it by 1 and return the list
                digits[i] = digits[i] + 1
                return digits
        # If all digits are 9, prepend 1 to the list
        return [1] + digits
```

Symmetric Tree

```
class Solution(object):
    def isMirror(self, left, right):
        if not left and not right:
            return True
        if not left or not right:
            return False
        return left.val == right.val and self.isMirror(left.left, right.right) and
self.isMirror(left.right, right.left)

    def isSymmetric(self, root):
        if not root:
            return True
        return self.isMirror(root.left, root.right)
```

Same Tree

```
lass Solution:
    def isSameTree(self, p: TreeNode, q: TreeNode) -> bool:
        if not p and not q:
            return True
        if not p or not q:
            return False
        return p.val == q.val and self.isSameTree(p.left, q.left) and
self.isSameTree(p.right, q.right)
```

Maximum Depth of Binary Tree

```
class Solution:
    def maxDepth(self, root):
        if not root:
```

```
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

Path Sum

```
# Definition for a  binary tree node
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    # @param root, a tree node
    # @param sum, an integer
    # @return a boolean
    # 1:27
    def hasPathSum(self, root, sum):
        if not root:
            return False

        if not root.left and not root.right and root.val == sum:
            return True

        sum -= root.val

        return self.hasPathSum(root.left, sum) or self.hasPathSum(root.right, sum)
```

Best Time to Buy and Sell Stock

```
class Solution:
    def maxProfit(self, prices):
        buy = prices[0]
        profit = 0
        for i in range(1, len(prices)):
            if prices[i] < buy:
                buy = prices[i]
            elif prices[i] - buy > profit:
                profit = prices[i] - buy
        return profit
```

Single Number

```
class Solution:
    def singleNumber(self, nums):
        index = 0
        for num in nums:
            index ^= num
        return index
```

Linked List Cycle

```python
class Solution:
    def hasCycle(self, head):

        slow = head
        fast = head

        while fast and fast.next:

            slow = slow.next
            fast = fast.next.next

            if slow == fast:
                return True

        return False
```

Excel Sheet Column Number

```python
class Solution(object):
    def titleToNumber(self, columnTitle):
        result = 0
        for char in columnTitle:
            result = result * 26 + (ord(char) - 64)
        return result
```